

UI

To start load testing, you can use the UI by running the following command:

```
bash ./start-ui.sh <rebuild>
```

The **rebuild** flag triggers a complete rebuild of all components.

Note: On the first run, the script will automatically build the UI if it hasn't been created yet before spawning the graphical interface. Once launched, you can configure and initiate load tests using the dashboard.



TrunkLoad
The OpenSearch Load Tester

OpenSearch Loadtester UI

Please select test-data generation document type:

Please select document type ▼

Please select the scenario, that should be executed:

Please enter scenario ▼

Please select test-data generation mode:

Please select test-data generation mode ▼

Please enter test-data generation count:

Enter count

Please enter test-data generation batch size:

Enter batch size

Please enter load-generator replica amount:

Enter replicas

Please enter metrics batch size:

Enter metrics batch size

Start Load Test

Kill container(s)

You can customize the following parameters for your load test:

- Data Generation Type: Choose between ANO or DUO.
- Load Scenario: Select the scenario to be executed by the load generator.
- Load Scenario: Select the scenario to be executed by the load generator.
- Data Strategy: Select whether to generate new data (Dynamic) or reuse previously generated data (Persistent).
- Testdata Generation Batch Size: Configure the batch size of testdata generation.
- Data Volume: Define the total amount of test data to be generated.
- Load Generators: Set the number of concurrent generator instances to start.
- Metrics Batch Size: Configure the batch size of metrics sent to Metrics Reporter.

Tip: If you select `custom-scenario.yaml`, you can visually design your own scenario using the graphical editor.

Please select the scenario, that should be executed:

custom-scenario.yaml

Schedule duration: e.g. PT30S

QPS: e.g. 20

Query response timeout e.g. PT1M

Warmup: select

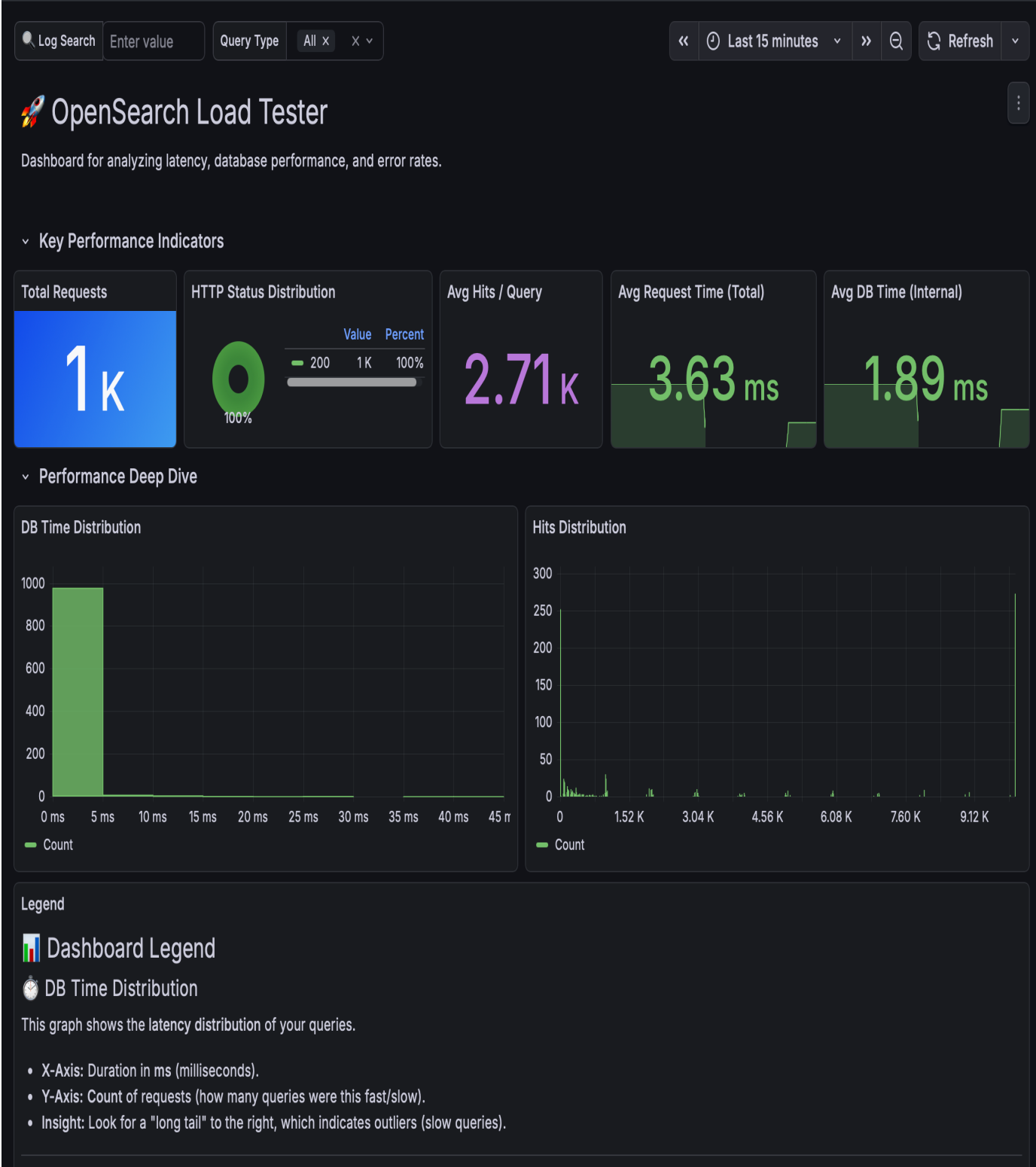
- ☐ ANO_PAYROLL_RANGE ☐ DUO_INVOICE_CATEGORY ☐ DUO_STATE_LOCATION
- ☐ DUO_BOOKING_BY_CLIENT_AND_STATE ☐ ANO_CLIENTS_AGGREGATION ☐ ANO_CLIENT_BY_YEAR
- ☐ ANO_DIS_MAX ☐ ANO_DIS_MAX_EXPENSIVE ☐ DUO_CLIENT_BY_CUSTOMER_NUMBER
- ☐ DUO_CLIENT_BY_NAME_AND_STATE ☐ ANO_PAYROLL_TYPE_LANGUAGE ☐ DUO_BOOKING_BY_COSTCENTER_AND_DATE
- ☐ DUO_BOOKING_BY_AMOUNT_RANGE ☐ DUO_INVOICE_DIS_MAX ☐ DUO_INVOICE_DIS_MAX_EXPENSIVE
- ☐ DUO_COMPLEX ☐ DOCNAME_REGEX ☐ ANO_MULTI_REGEX
- ☐ DUO_MULTI_REGEX ☐ ANO_SPAN_NEAR ☐ DUO_SPAN_NEAR
- ☐ ANO_MORE_LIKE_THIS ☐ DUO_MORE_LIKE_THIS ☐ ANO_PREFIX_RANGE
- ☐ DUO_MULTI_PREFIX_SORT ☐ DUO_DATE_RANGE ☐ ANO_PREFIX_MATCH

Execution and Results

Clicking the "Start Load Test" button triggers the build process for all required Docker images and launches the complete OpenSearch load-tester stack. You can view the results in Grafana:

URL: `http://localhost:3000`

Path: Dashboards → Load Testing → OpenSearch Load Test



Make

For use on a system without a graphical interface, you may use the **Makefile**. The steps below describe how to initiate and control this process.

First ensure that all required tools are installed. To deploy the full stack, start the interactive setup with:

```
make loadtest
```

The setup prompts for:

- how many load generators should be started
- how many test files should be uploaded to the OpenSearch instance
- what type of test data should be generated (ANO/DUO)

During this interaction, several tasks run automatically in the background:

```
make clean
make build
make run
```

These commands clean up the environment, build the Docker images, and start the complete system including OpenSearch, Alloy, Loki, and Grafana.

Viewing logs

All container logs can be viewed with:

```
make logs
```

This attaches to the logs of every container defined in `docker-compose.yml`. To view logs of a specific container, use:

```
make logs <container-name>
```

Resource usage

To display CPU and memory consumption of all running containers, use:

```
make status
```

Available Make Commands

Make command	Description
make build	Builds all Docker images
make run	Starts all containers
make stop	Stops all containers
make logs	Shows all container logs
make logs testdata-generator	Show logs for the <code>testdata-generator</code> container
make logs load-generator	Show logs for the <code>load-generator</code> container
make logs metrics-reporter	Show logs for the <code>metrics-reporter</code> container
make logs test-target-opensearch	Show logs for the <code>test-target-opensearch</code> container
make logs loki	Show logs for the <code>loki</code> container
make logs alloy	Show logs for the <code>alloy</code> container
make logs grafana	Show logs for the <code>grafana</code> container
make loadtest	Interactive load test setup
make status	Shows CPU and memory usage of running containers

Make command	Description
make clean	Stops containers and removes volumes/images
make help	Shows this help message

Convenience commands

For common tasks such as building images, stopping containers, or cleaning up the environment, use:

```
make build
make stop
make clean
```

Environment Variables

This page documents all environment variables used in the OpenSearch Load Tester project. Following are the environment variables used in the OpenSearch Load Tester project. Variables are grouped by service:

- TestDataGenerator
- LoadGenerator
- MetricsReporter
- OpenSearch

Each variable includes its default value, description, and when it should be changed.

TestDataGenerator – Environment Variables

These variables configure how test data is generated and preloaded into OpenSearch.

Table: TestDataGenerator Variables

Variable	Default	Description	Usage
SPRING_APPLICATION_NAME	testdata-generator	Name of the Spring Boot application.	Mainly used for logging and identification.
OPENSEARCH_URL	http://test-target-opensearch:9200	URL of the OpenSearch instance to index test data into.	Change if OpenSearch runs on a different host or port.
DATA_GENERATION_MODE	dynamic	Defines how documents are generated.	dynamic generates data at runtime; persistent enables reuse.
DATA_GENERATION_DOCUMENT_TYPE	ano	Type of document to generate.	Use ano or duo depending on the target index schema.

Variable	Default	Description	Usage
DATA_GENERATION_COUNT	1000	Total number of documents to generate.	Increase to preload larger datasets.
DATA_GENERATION_BATCH_SIZE	1000	Number of documents per bulk indexing request.	Reduce if OpenSearch returns 413/429 errors.
DATA_GENERATION_OUTPUT_PATH	data/testdata.json	File path for persisted test data (persistent mode only).	Change to store datasets in a custom location.
LOGGING_LEVEL_ROOT	ERROR	Global logging level.	Set to INFO or DEBUG for troubleshooting.
LOGGING_LEVEL_COM_OPENSEARCHLOADTESTER_TESTDATAGENERATOR	INFO	Logging level for the Test Data Generator package.	Increase to DEBUG for detailed generation and indexing logs.

LoadGenerator – Environment Variables

These variables define how queries are executed and how performance metrics are sent to the MetricsReporter.

Table: LoadGenerator Variables (application properties)

Variable	Default	Description	Usage
OPENSEARCH_URL	http://test-target-opensearch:9200/	URL of the OpenSearch instance the LoadGenerator queries.	Change if OpenSearch runs elsewhere.
LOGGING_LVL_LOAD_GEN	INFO	Logging for Load Generator package	Change to DEBUG when troubleshooting.
LOAD_GENERATOR_REPLICAS	1	Expected number of load-generator replicas.	Bump when scaling out generators.
METRICS_REPORTER_URL	http://metrics-reporter:8080/api/	Base URL for metrics submission.	Modify if MetricsReporter runs on another host/port.
METRICS_BATCH_SIZE	1000	Number of metrics to batch before sending to MetricsReporter.	Increase to reduce network overhead at high QPS.
SCENARIO_CONFIG_PATH	ano-low-scenario.yaml	Path to the scenario YAML file used by the LoadGenerator.	Override to change which scenario is executed.

MetricsReporter – Environment Variables

These variables control how metrics are aggregated and exposed.

Table: MetricsReporter Variables

Variable	Default	Description	Usage
----------	---------	-------------	-------

Variable	Default	Description	Usage
SPRING_APPLICATION_NAME	metrics-reporter	Application name.	For logs/dashboards.
LOAD_GENERATOR_REPLICAS	1	Expected number of load-generator replicas.	Bump when scaling out generators.
REPORT_OUTPUT_DIRECTORY	./reports	Directory where reports are written.	Point to persistent storage if needed.
REPORT_NDJSON_FILENAME	tmp_query_results.ndjson	Temporary NDJSON file for streaming writes.	Usually unchanged.
REPORT_RESULTSJSON_FILENAME	query_results.json	Final JSON array of all query results.	Change to avoid collisions per run.
REPORT_STATS_FILENAME	statistics.json	Aggregated statistics JSON file.	Change to avoid collisions per run.
LOGGING_LEVEL_ROOT	ERROR	Global logging level.	Set to INFO/DEBUG when debugging.
LOGGING_LEVEL_COM_OPENSEARCHLOADTESTER_METRICSREPORTER	INFO	Logging for Metrics Reporter package.	Increase to DEBUG for troubleshooting.

Report output location and cleanup

- Local runs (default `application.properties`): output directory is `../reports` relative to the metrics-reporter module.
- Docker compose: `REPORT_OUTPUT_DIRECTORY` defaults to `./reports` inside the container and is backed by the named volume `reports`.
- On startup, the Metrics Reporter deletes existing report files (`tmp_query_results.ndjson`, `query_results.json`, `statistics.json`) in the output directory before starting a new run.

OpenSearch – Environment Variables

These variables configure the embedded OpenSearch test cluster.

Table: OpenSearch Target Variables

Variable	Default	Description	Usage
DISABLE_SECURITY_PLUGIN	true	Disables authentication and TLS.	Keep true for local development/testing.
discovery.type	single-node	Starts OpenSearch as a single-node cluster.	Change only for multi-node setups.

Variable	Default	Description	Usage
OPENSEARCH_JAVA_OPTS	-Xms512m -Xmx512m	JVM heap size.	Increase for heavy loads or large datasets.

Common Logging Variables

The following variables may be used across multiple services:

Variable	Description
LOGGING_LVL_TESTDATA_GEN	Logging level for TestDataGenerator.
LOGGING_LVL_LOAD_GEN	Logging level for LoadGenerator.
LOGGING_LVL_METRICS_REPORTER	Logging level for MetricsReporter.
GRAFANA_LOGGING_LEVEL	Logging level for Grafana.

These allow centralized configuration via a `.env` file.

How to Override Environment Variables

Using a `.env` file

```
OPENSEARCH_URL=http://localhost:9200
DATA_GENERATION_COUNT=5000
LOGGING_LVL_ROOT=INFO
```

Overriding inline

```
docker compose run -e DATA_GENERATION_COUNT=20000 testdata-generator
```

Overriding inside `docker-compose.yml`

```
environment:
  - DATA_GENERATION_COUNT=20000
```

YAML Scenario Configuration

The **Load Generator** supports scenario-based configuration using YAML files. Each scenario YAML file defines how a load test behaves: duration, QPS, concurrency, and the type of queries executed against OpenSearch.

Scenarios are stored in:

```
./load-generator/src/main/resources/scenarios
```

The scenario used at runtime is selected via:

```
SCENARIO_CONFIG
```

Default:


```
ano-low-scenario.yaml
```

1. Purpose of Scenario YAML Files

A Scenario YAML file allows users to:

- Define **how long** a test should run
 - Control **queries per second (QPS)**
 - Configure **warm-up** and **ramp-up** phases
 - Adjust **concurrency parameters** (clients, threads)
 - Specify **which query type** the LoadGenerator should execute
 - Pass **query parameters** (e.g., year ranges)
-

2. Scenario File Structure

Below is the default scenario configuration included in the project:

```
---
name: ano-low-scenario
document_type: ANO
schedule_duration: PT10S
queries_per_second: 100
query_response_timeout: PT30S
enable_warm_up: true
query_mix:
  - ANO_PAYROLL_RANGE
  - ANO_CLIENTS_AGGREGATION
  - ANO_CLIENT_BY_YEAR
  - ANO_PAYROLL_TYPE_LANGUAGE
```

To configure percentages for each query type use the following format:

```
---
name: ano-medium-scenario
document_type: ANO
schedule_duration: PT30S
queries_per_second: 200
query_response_timeout: PT60S
enable_warm_up: true
query_mix:
  - type: ANO_PAYROLL_RANGE
    percent: 20
  - type: ANO_CLIENTS_AGGREGATION
    percent: 10
  - type: ANO_CLIENT_BY_YEAR
    percent: 5
  - type: ANO_PAYROLL_TYPE_LANGUAGE
    percent: 5
  - type: ANO_MULTI_REGEX
    percent: 30
  - type: DOCNAME_REGEX
    percent: 30
```

3. Field-by-Field Explanation

This table explains each component of the scenario YAML.

Scenario Metadata

Field	Example	Allowed Values / Options	Explanation
name	ano-low-scenario.yaml	Any text (string)	Human-readable scenario identifier.
document_type	ANO	ANO, DUO	Specifies which dataset/index the scenario targets. Must match what the TestDataGenerator created.
schedule_schedule_duration	PT10S	ISO-8601 durations: • PT10S (10s) • PT30S (30s) • PT1M (1 min) • PT5M (5 min) • PT30M, PT1H	Total duration of the load test.
queries_per_second	200	Any positive integer	Target QPS the generator tries to maintain.
query_response_timeout	PT60S	ISO-8601 duration	Per-request client timeout.
enable_warm_up	true	true false	Enable a warm-up phase before the main run.
query_mix	see example	List of entries: either a string type or object { type: <NAME>, percent: <INT> }. Percent sum to 100	Weighted query selection; percent controls relative frequency of each type.

The parameters section depends on the selected query type.

4. Changing the Scenario File

You can create multiple scenario files. Then select the desired scenario using:

Option A — .env file

```
SCENARIO_CONFIG=ano-high-scenario.yaml
```

Option B — Inline override

```
SCENARIO_CONFIG=ano-high-scenario.yaml docker compose up load-generator
```

Option C — Modify docker-compose

```
environment:  
  - SCENARIO_CONFIG=ano-high-scenario.yaml
```

5. Creating Your Own Scenario (Step-by-Step)

1. Create a new .yaml file in:

```
./load-generator/src/main/resources/scenarios
```

2. Add your scenario fields.

3. Select it using:

```
SCENARIO_CONFIG=<your-scenario.yaml>
```

4. Start the system:

```
make build && make run
```
